Python & Bash for DevOps — Revision Notebook

Aryan

August 2026

Contents

How to Use This Notebook

Flow: Learn → Practice → Build → Revise → Interview.

- [MUST KNOW] **MUST KNOW** — you will use this weekly on the job.
- [GOOD TO KNOW] **GOOD TO KNOW** — comes up often enough to learn properly.
- [ADVANCED] **ADVANCED** — nice to have, not urgent.

Every major section ends with **Quick Revision**, **Must Remember**, and **Practice**.

PART 1 — PYTHON FOR DEVOPS

1. Python Fundamentals [MUST KNOW]

1.1 Installation & Interpreter

- Check version: `python3 --version`
- Interactive shell: `python3` (REPL) — good for quick tests, not for real scripts.
- Run a script: `python3 script.py`
- Make executable on Linux: add shebang `#!/usr/bin/env python3`, then `chmod +x script.py`.

Common mistake: using `python` instead of `python3` on systems where `python` isn't mapped (common on fresh Ubuntu/EC2 AMIs) → `command not found`.

1.2 Variables

```
name = "aryan"
count = 5
ratio = 0.75
is_ready = True
```

- Dynamically typed — no declared type, type is inferred at runtime.
- Naming: `snake_case` for variables/functions, `UPPER_CASE` for constants.

1.3 Data Types

Type	Example	Notes
`int`	`10`	arbitrary precision
`float`	`3.14`	
`str`	`"ec2"`	immutable
`bool`	`True/False`	subclass of int
`NoneType`	`None`	absence of value
`list`	`[1,2,3]`	mutable, ordered
`tuple`	`(1,2)`	immutable, ordered
`dict`	`{"k":"v"}`	key-value
`set`	`{1,2,3}`	unique, unordered

Check type: `type(x)`, isinstance check: `isinstance(x, int)`.

1.4 Type Conversion

```
int("42")      # str -> int
str(42)        # int -> str
float("3.5")
list("abc")    # ['a','b','c']
```

Common mistake: `int("3.5")` raises `ValueError` — must go through `float()` first: `int(float("3.5"))`.

1.5 Operators

- Arithmetic: `+ - * / // % **`
- Comparison: `== != > < >= <=`
- Logical: `and or not`
- Assignment: `= += -= *= /=`
- Membership: `in`, `not in`
- Identity: `is`, `is not` (identity, not equality — use for `None` checks: `if x is None`)

Common mistake: using `==` to check `None` instead of `is None`.

1.6 Input / Output

```
name = input("Enter name: ")
print("Hello", name, sep=", ", end="\n")
```

1.7 Comments

```
# single line
"""
multi-line
docstring/comment
"""
```

1.8 Indentation

Python uses indentation (4 spaces, no tabs) instead of {} to define blocks. Mixing tabs/spaces causes `IndentationError` — configure your editor to insert spaces.

1.9 Strings & String Methods

```
s = "  Server-01  "
s.strip()          # "Server-01"
s.lower(); s.upper()
s.split("-")        # ["  Server", "01  "]
"-".join(["a","b"]) # "a-b"
s.replace("Server", "Node")
s.startswith("  Se")
"01" in s
len(s)
```

1.10 f-strings [MUST KNOW]

```
host = "web01"
cpu = 87.5
print(f"{host} CPU usage: {cpu:.1f}%")
print(f"{host=}")          # debug print: host='web01'
```

Preferred over `%` formatting or `.format()` — cleaner and faster.

Quick Revision

- Dynamic typing, indentation-based blocks, f-strings for formatting.
- `is` for identity/None checks, == for value equality.

Must Remember

1. `python3`, not `python`.
2. f-strings for all string formatting.
3. `is None` not `== None`.
4. Strings are immutable — methods return new strings.
5. 4-space indentation, never mix tabs/spaces.

Practice

1. Write a script that reads a hostname from input and prints it uppercase with `f-string`.
2. Convert "512" (MB) to GB as a float, printed to 2 decimals.
3. Strip and split a comma-separated string of IPs into a list.

---

2. Python Data Structures [MUST KNOW]

2.1 Lists

```
servers = ["web01", "web02", "db01"]
servers.append("web03")
servers.remove("db01")
servers[0]              # "web01"
servers[-1]              # last element
servers[1:3]              # slicing
len(servers)
sorted(servers)
servers.sort(reverse=True)
```

2.2 Tuples

```
region = ("us-east-1", "us-east-1a")   # immutable — use for fixed records
az, zone = region                      # unpacking
```

Use tuples for data that shouldn't change (e.g., a function returning `(status_code, body)`).

2.3 Sets

```
running = {"web01", "web02", "db01"}
expected = {"web01", "web02", "web03"}
running - expected   # {'db01'}  -> extra/unexpected servers
expected - running   # {'web03'} -> missing servers
running & expected   # intersection
```

DevOps use case: diffing "expected infra" vs "actual infra" (drift detection).

2.4 Dictionaries [MUST KNOW]

```
instance = {"id": "i-0abc123", "state": "running", "type": "t3.micro"}
instance["state"]
instance.get("tags", {})      # safe access with default
instance["az"] = "us-east-1a" # add/update key
for key, value in instance.items():
    print(key, value)
```

2.5 Nested Data Structures

```
instances = [
    {"id": "i-1", "state": "running", "tags": {"env": "prod"}},
    {"id": "i-2", "state": "stopped", "tags": {"env": "dev"}},
]
prod = [i["id"] for i in instances if i["tags"]["env"] == "prod"]
```

This shape (list of dicts) is exactly what `boto3` and most cloud/API responses return.

2.6 Comprehensions [MUST KNOW]

```
# list comprehension
running_ids = [i["id"] for i in instances if i["state"] == "running"]

# dict comprehension
id_to_state = {i["id"]: i["state"] for i in instances}

# set comprehension
envs = {i["tags"]["env"] for i in instances}
```

2.7 Slicing

```
log_lines[-10:]     # last 10 lines
log_lines[:5]        # first 5 lines
log_lines[::2]       # every 2nd line
```

2.8 Iteration Helpers

```
for i, s in enumerate(servers, start=1):
    print(i, s)

for s, r in zip(servers, regions):
    print(s, r)
```

2.9 Useful Built-ins

`len()`, `sum()`, `min()`, `max()`, `sorted()`, `any()`, `all()`, `map()`, `filter()`

```python
cpu_values = [45, 92, 30, 88]
any(c > 90 for c in cpu_values)   # True -> alert
all(c < 95 for c in cpu_values)
```

Common mistake: modifying a list while iterating over it. Iterate over a copy: `for x in list(servers):`.

Quick Revision

- Lists = ordered/mutable, tuples = ordered/immutable, sets = unique/unordered, dicts = key-value.
- Comprehensions are the idiomatic way to transform/filter data.
- List of dicts is the standard shape for API/boto3 responses.

Must Remember

1. `dict.get(key, default)` avoids KeyError.
2. Set difference (`-`) is the go-to pattern for drift/diff checks.
3. List comprehension > manual `for` + `append()` loop for simple transforms.
4. Don't mutate a list while looping over it directly.
5. `enumerate()` and `zip()` remove the need for manual index tracking.

Practice

1. Given a list of instance dicts, extract IDs of all `stopped` instances using a list comprehension.
2. Build a dict mapping instance id → tags using a dict comprehension.
3. Find servers present in `desired_state` but missing from `actual_state` using sets.

---

3. Conditions and Loops [MUST KNOW]

3.1 if / elif / else

```python
cpu = 92
if cpu > 90:
    status = "CRITICAL"
elif cpu > 70:
    status = "WARNING"
else:
    status = "OK"
```

3.2 for Loop

```python
for server in servers:
    print(f"Checking {server}")
```

3.3 while Loop

```
retries = 0
while retries < 3:
    success = attempt_connection()
    if success:
        break
    retries += 1
```

3.4 break / continue / pass

```
for server in servers:
    if server == "maintenance-01":
        continue      # skip this one
    if server == "stop-here":
        break          # exit loop entirely
    pass               # placeholder, does nothing
```

3.5 range()

```
for i in range(5):        # 0..4
    pass
for i in range(1, 6):     # 1..5
    pass
for i in range(0, 10, 2): # step 2
    pass
```

3.6 enumerate() and zip()

```
for idx, server in enumerate(servers):
    print(idx, server)

for server, status in zip(servers, statuses):
    print(server, status)
```

DevOps-style example

```
disks = {"/": 45, "/var": 92, "/data": 78}
for mount, used_percent in disks.items():
    if used_percent >= 90:
        print(f"ALERT: {mount} is at {used_percent}% usage")
    elif used_percent >= 75:
        print(f"WARNING: {mount} is at {used_percent}% usage")
```

Quick Revision

- `elif` chains for threshold-based alerting (very common pattern).
- `while` + retry counters for connection/health-check retries.
- `break`/`continue` for early exits/skips inside monitoring loops.

Must Remember

1. Threshold-based `if/elif/else` is the backbone of monitoring scripts.
2. `while` loops with a max-retry counter avoid infinite loops.
3. `range(start, stop, step)` — `stop` is exclusive.
4. `enumerate()` instead of manual counters.
5. `zip()` to iterate two related lists together.

Practice

1. Write a loop that prints WARNING/CRITICAL based on CPU thresholds for a list of values.
2. Write a retry loop (max 5 tries) simulating a flaky connection with `random`.
3. Use `zip()` to pair server names with their IPs and print both.

---

4. Functions [MUST KNOW]

4.1 Defining Functions

```python
def greet(name):
    return f"Hello, {name}"
```

4.2 Parameters, Defaults, Keyword Args

```python
def check_disk(mount, threshold=90):
    return f"Checking {mount} at {threshold}%"

check_disk("/data")                    # uses default threshold
check_disk("/data", threshold=80)      # keyword arg
check_disk(mount="/data", threshold=80)
```

4.3 *args and **kwargs

```python
def run_command(*args, **kwargs):
    print(args)      # tuple of positional args
    print(kwargs)     # dict of keyword args

run_command("ls", "-la", cwd="/tmp", timeout=5)
```

DevOps use case: wrapping `subprocess.run()` so your helper accepts any extra options.

4.4 Return Values

```python
def get_status(cpu):
    if cpu > 90:
        return "CRITICAL", cpu
    return "OK", cpu

status, value = get_status(95)   # multiple return values via tuple
```

4.5 Lambda Functions

```
square = lambda x: x * x
sorted(servers, key=lambda s: s["cpu"], reverse=True)
```

Use lambdas only for short, throwaway functions (e.g., as key= in `sorted()`/`filter()`). For anything with logic, use a named `def`.

4.6 Scope

```
threshold = 90   # global

def check(cpu):
    # local scope; reading global is fine
    return cpu > threshold

def update_threshold(new_value):
    global threshold   # needed to REASSIGN a global inside a function
    threshold = new_value
```

Common mistake: forgetting `global` when trying to modify a global variable inside a function (it silently creates a local variable instead).

4.7 Recursion

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Rarely needed in DevOps scripting — mentioned for completeness. Prefer iteration for things like directory traversal (`os.walk` already handles recursion for you).

Quick Revision

- Functions are the unit of reusable automation logic — every script should be broken into small functions.
- `*args`/`**kwargs` let wrapper functions forward arbitrary arguments.
- `global` needed only when reassigning (not reading) a global inside a function.

Must Remember

1. Always give functions a clear single responsibility.
2. Default args for optional thresholds/config (`threshold=90`).
3. Return tuples for multiple values, unpack on the caller side.
4. Lambdas only for one-liners (mainly as key=).
5. Avoid mutable default arguments (`def f(items=[])`) — this is a classic bug (the list persists across calls).

Practice

1. Write `check_service(name, expected_state="running")` returning a bool.

2. Write a function using *args to accept any number of hostnames and ping-check each.
3. Write a function with a mutable default argument bug, then fix it.

---

5. Exception Handling [MUST KNOW]

5.1 try / except / else / finally

```
import subprocess

try:
    result = subprocess.run(["systemctl", "status", "nginx"], check=True, capture_output=T
except subprocess.CalledProcessError as e:
    print(f"Service check failed: {e}")
else:
    print("Service is healthy")
finally:
    print("Check complete")   # always runs — good for cleanup/logging
```

5.2 Catching Specific Exceptions

```
try:
    value = int(config["timeout"])
except KeyError:
    print("Missing 'timeout' key in config")
except ValueError:
    print("'timeout' is not a valid number")
```

Common mistake: bare `except:` catches everything including `KeyboardInterrupt`/`SystemExit` — always catch specific exceptions, or at minimum `except Exception as e:`.

5.3 raise

```
def validate_port(port):
    if not (1 <= port <= 65535):
        raise ValueError(f"Invalid port: {port}")
```

5.4 Custom Exceptions

```
class DeploymentError(Exception):
    """Raised when a deployment step fails."""
    pass

def deploy(version):
    if not version:
        raise DeploymentError("Version cannot be empty")
```

5.5 Exception Handling in Automation Scripts

```python
def backup_file(src, dst):
    try:
        shutil.copy2(src, dst)
    except FileNotFoundError:
        logging.error(f"Source file not found: {src}")
        return False
    except PermissionError:
        logging.error(f"Permission denied copying {src} -> {dst}")
        return False
    except Exception as e:
        logging.exception(f"Unexpected error backing up {src}: {e}")
        return False
    return True
```

Automation scripts should **never crash silently or unhandled** — catch expected failure modes, log them, and either retry, skip, or exit with a non-zero code.

Quick Revision

- `try/except/else/finally` — `else` runs only if no exception, `finally` always runs.
- Catch specific exceptions, not bare `except:`.
- Custom exceptions make automation errors self-documenting.

Must Remember

1. Never use a bare `except:` in production scripts.
2. `finally` is ideal for cleanup (closing connections, temp files).
3. Raise custom exceptions for domain-specific automation errors.
4. Log exceptions (`logging.exception`) instead of just printing.
5. Failed automation should exit with a non-zero exit code (`sys.exit(1)`), so CI/CD and cron detect failure.

Practice

1. Write a function that reads a config file and raises a custom `ConfigError` if a required key is missing.
2. Wrap a `subprocess.run()` call with proper exception handling and logging.
3. Write a retry-with-exception-handling loop for a flaky network call. # 6. Files and Directories [MUST KNOW]

6.1 Reading & Writing Files

```python
with open("app.log") as f:          # read mode default, auto-closes file
    content = f.read()

with open("app.log") as f:
    for line in f:                   # memory-efficient line-by-line
        print(line.strip())

with open("output.txt", "w") as f:   # overwrite
    f.write("deployment complete\n")
```

```
with open("output.txt", "a") as f:   # append
    f.write("another line\n")
```

Always use `with` — it guarantees the file is closed even if an exception occurs. Never manually `open()`/`close()` in new code.

6.2 pathlib (modern, preferred) [MUST KNOW]

```
from pathlib import Path

p = Path("/var/log/app.log")
p.exists()
p.is_file()
p.suffix          # ".log"
p.stem            # "app"
p.parent          # PosixPath('/var/log')
p.name             # "app.log"

Path("/opt/backups").mkdir(parents=True, exist_ok=True)
for f in Path("/var/log").glob("*.log"):
    print(f)
for f in Path("/var/log").rglob("*.log"):   # recursive
    print(f)
```

6.3 os and shutil

```
import os, shutil

os.getcwd()
os.listdir("/etc")
os.path.join("/opt", "app", "config.yml")
os.path.exists("/etc/nginx")
os.remove("/tmp/old.log")            # delete a file
os.rmdir("/tmp/empty_dir")           # delete empty dir
shutil.rmtree("/tmp/full_dir")       # delete dir + contents
shutil.copy2("a.txt", "b.txt")       # copy with metadata
shutil.move("a.txt", "/backup/a.txt")
shutil.disk_usage("/")               # (total, used, free) in bytes
```

6.4 File Permissions

```
import os, stat

os.chmod("deploy.sh", 0o755)
mode = os.stat("deploy.sh").st_mode
oct(stat.S_IMODE(mode))   # '0o755'
```

6.5 Practical Examples

```python
# Reading a config file
from pathlib import Path
cfg_path = Path("/etc/myapp/config.yml")
if not cfg_path.exists():
    raise FileNotFoundError(f"Config not found: {cfg_path}")

# Finding files
log_files = list(Path("/var/log").rglob("*.log"))

# Checking whether a file exists before acting
if Path("/tmp/lockfile").exists():
    print("Job already running, exiting")
```

Quick Revision

- `pathlib.Path` is the modern, cross-platform way to handle paths — prefer over `os.path` for new code.
- Always use `with open(...)`.
- `shutil` for high-level copy/move/delete of files & directory trees.

Must Remember

1. `with open(...) as f:` — always.
2. `Path.mkdir(parents=True, exist_ok=True)` avoids `FileExistsError`.
3. `shutil.rmtree()` deletes non-empty directories — `os.rmdir()` only deletes empty ones.
4. Check `Path.exists()` before operating on files in automation.
5. `glob()` = current dir only, `rglob()` = recursive.

Practice

1. Write a script that finds all `.log` files older than 7 days under `/var/log` and lists them.
2. Write a function that safely creates a backup directory structure.
3. Read a YAML-looking config file line by line and print only non-comment lines.

---

7. OS and System Automation [MUST KNOW] (VERY IMPORTANT)

This is the core of DevOps scripting — using Python to drive the Linux system itself.

7.1 os vs subprocess vs shutil — When to Use What

Need	Use
Read env vars, get PID, path joins	`os`
Run a shell command / external program	`subprocess`
High-level file/dir copy, move, delete, disk usage	`shutil`
OS/platform detection (Linux vs Windows)	`platform`

Need	Use
Command-line args, exit codes	sys

Rule of thumb: if you need the *output* or *exit status* of a command (`df -h`, `systemctl status`), use subprocess. Don't shell out for things Python can do natively (e.g., don't `subprocess.run(["ls"])` — use `os.listdir()`/`pathlib`).

7.2 subprocess — running Linux commands [MUST KNOW]

```
import subprocess

# Preferred modern API: subprocess.run()
result = subprocess.run(
    ["df", "-h"],
    capture_output=True,
    text=True,        # decode bytes to str
    check=True          # raise CalledProcessError on non-zero exit
)
print(result.stdout)
print(result.returncode)
```

NEVER use `shell=True` with untrusted/user-supplied input — it enables shell injection. Pass a list of args, not a single string, whenever possible.

```
# BAD (shell injection risk if hostname comes from user input)
subprocess.run(f"ping -c 1 {hostname}", shell=True)

# GOOD
subprocess.run(["ping", "-c", "1", hostname])
```

7.3 Practical System Checks

```
import subprocess

def run(cmd):
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.returncode, result.stdout.strip(), result.stderr.strip()

# Disk usage
code, out, err = run(["df", "-h"])

# Service status
code, out, err = run(["systemctl", "is-active", "nginx"])
is_running = out == "active"

# Restart a service
run(["systemctl", "restart", "nginx"])

# Memory usage
code, out, err = run(["free", "-m"])
```

```
# CPU load average
with open("/proc/loadavg") as f:
    load1, load5, load15 = f.read().split()[:3]

# Running processes
code, out, err = run(["ps", "aux"])
```

Using the `psutil` library (cleaner than parsing command output) [GOOD TO KNOW]

```
import psutil

psutil.cpu_percent(interval=1)
psutil.virtual_memory().percent
psutil.disk_usage("/").percent
[p.info for p in psutil.process_iter(["pid", "name"])]
```

`psutil` is not in the standard library (`pip install psutil`) but is the industry-standard choice for system metrics in Python — avoids fragile text-parsing of command output.

7.4 Environment Variables

```
import os

aws_region = os.environ.get("AWS_REGION", "us-east-1")   # safe, with default
os.environ["DEBUG"] = "1"                                  # set for this process
"AWS_ACCESS_KEY_ID" in os.environ                            # check existence
```

Common mistake: `os.environ["KEY"]` raises `KeyError` if missing — use `.get()` with a default in scripts that shouldn't crash.

7.5 Exit Codes

```
import sys

if not deployment_successful:
    sys.exit(1)     # non-zero = failure, critical for CI/CD & cron
sys.exit(0)          # success (implicit if script ends normally)
```

Exit codes matter enormously in DevOps: CI/CD pipelines, cron jobs, and shell `&&`/`||` chains all rely on them.

7.6 Process Management

```
import subprocess

# Run in background (non-blocking)
proc = subprocess.Popen(["python3", "worker.py"])
proc.pid
proc.poll()          # None if still running, else return code
proc.terminate()
proc.wait(timeout=10)
```

Quick Revision

- `subprocess.run(list, capture_output=True, text=True, check=True)` is the standard pattern.
- Never `shell=True` with variable input.
- `psutil` for clean, reliable system metrics instead of parsing `top`/`free` output.
- Exit codes are the contract between your script and CI/CD/cron.

Must Remember

1. Prefer list-form args to `subprocess`, never `shell=True` with user input.
2. `capture_output=True, text=True` to get decoded stdout/stderr.
3. `check=True` raises on failure — or check `returncode` manually.
4. `os.environ.get(key, default)` over `os.environ[key]`.
5. `sys.exit(1)` on failure so orchestration tools can detect it.

Practice

1. Write a function `service_is_active(name)` using `subprocess`.
2. Write a script that checks disk usage via `shutil.disk_usage` and exits 1 if any mount > 90%.
3. Write a script that reads `AWS_PROFILE` from env vars, defaulting to `"default"`.

---

8. JSON, YAML and Configuration [MUST KNOW]

8.1 JSON

```
import json

data = {"service": "nginx", "port": 80, "enabled": True}

json_str = json.dumps(data, indent=2)          # dict -> JSON string
parsed = json.loads(json_str)                   # JSON string -> dict

with open("config.json") as f:
    config = json.load(f)                       # read JSON file -> dict

with open("output.json", "w") as f:
    json.dump(data, f, indent=2)                # write dict -> JSON file
```

DevOps use case: parsing API responses (`response.json()`), Terraform JSON output (`terraform show -json`), AWS CLI `--output json`.

8.2 YAML (PyYAML) [MUST KNOW]

```
import yaml   # pip install pyyaml

with open("docker-compose.yml") as f:
```

```
    config = yaml.safe_load(f)                    # ALWAYS use safe_load, never load()

with open("output.yml", "w") as f:
    yaml.safe_dump(config, f, default_flow_style=False)
```

Common mistake: `yaml.load()` without a Loader can execute arbitrary Python objects embedded in the YAML — a security risk. Always use `yaml.safe_load()`.

8.3 Config file patterns

```
import yaml
from pathlib import Path

def load_config(path="config.yml"):
    p = Path(path)
    if not p.exists():
        raise FileNotFoundError(f"Config not found: {path}")
    with open(p) as f:
        return yaml.safe_load(f)

config = load_config()
db_host = config.get("database", {}).get("host", "localhost")
```

8.4 Parsing API Responses

```
import requests

resp = requests.get("https://api.example.com/status")
data = resp.json()                # already parses JSON for you
status = data.get("status")
```

Quick Revision

- JSON: `json.dumps/loads` (strings), `json.dump/load` (files).
- YAML: always `yaml.safe_load` / `yaml.safe_dump`.
- `.get()` with defaults for safe nested config access.

Must Remember

1. `safe_load` not `load` for YAML — security.
2. `json.dump/load` work with file objects; `dumps/loads` work with strings.
3. `response.json()` from `requests` parses JSON directly.
4. Use `.get()` chains (or `dict.get(k, {})`) to avoid `KeyError` on nested config.
5. `indent=2` when writing JSON/YAML for human-readable output.

Practice

1. Write a script that loads a YAML config, validates required keys exist, and raises otherwise.
2. Convert a JSON file to YAML and vice versa.
3. Parse a mock API JSON response and extract a nested field safely.

---

9. Regular Expressions [GOOD TO KNOW]

9.1 Basics with re

```
import re

re.search(r"error", line)              # find first match anywhere -> Match or None
re.match(r"^\d+", line)                 # match only at start of string
re.findall(r"\d+\.\d+\.\d+\.\d+", text) # all IP-like matches
re.sub(r"password=\S+", "password=***", line)  # substitute (redact secrets)
```

9.2 Groups

```
m = re.search(r"(\d+\.\d+\.\d+\.\d+):(\d+)", "connection from 10.0.0.5:8080")
if m:
    ip, port = m.group(1), m.group(2)
```

9.3 Practical Patterns

Use case	Pattern
IPv4 address	`r"\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b"`
Log timestamp (ISO)	`r"\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}"`
Semantic version	`r"v?\d+\.\d+\.\d+"`
URL	`r"https?://[^\s]+"`
Log level	`r"\b(ERROR|WARN|INFO|DEBUG)\b"`

```
# Log parsing example
log_line = "2026-08-13T10:22:31 ERROR Failed to connect to 10.0.0.5:5432"
m = re.search(r"^(\S+) (ERROR|WARN|INFO) (.+)$", log_line)
timestamp, level, message = m.groups()

# Count errors in a log file
error_count = sum(1 for line in open("app.log") if re.search(r"\bERROR\b", line))
```

Common mistake: overusing regex for structured data (JSON/YAML/CSV) — parse those with their proper module instead. Regex is for unstructured text like logs.

Quick Revision

- `search` = anywhere, `match` = start only, `findall` = all matches, `sub` = replace.
- Use raw strings `r"..."` for regex patterns to avoid escaping issues.
- Groups `()` extract sub-parts of a match.

Must Remember

1. Raw strings (`r"..."`) for all regex patterns.
2. `re.sub` is great for redacting secrets in logs before printing/storing.
3. Don't regex-parse JSON/YAML — use `json`/`yaml` modules.
4. `findall` returns strings (or tuples if multiple groups), `search`/`match` return `Match` objects (or `None`).

5. Always check for None before calling `.group()`.

Practice

1. Extract all IP addresses from a log file.
2. Write a regex to redact any `password=...` or `token=...` values from a log line.
3. Parse a version string like `v1.4.2` into major/minor/patch using groups.

---

10. Modules and Packages [MUST KNOW]

10.1 import

```
import os
import subprocess as sp
from pathlib import Path
from datetime import datetime, timedelta
```

10.2 Creating Your Own Modules

```
myproject/
├── main.py
├── utils/
│   ├── __init__.py
│   ├── aws_helpers.py
│   └── logging_helpers.py
```

```
# main.py
from utils.aws_helpers import list_running_instances
```

10.3 pip, requirements.txt, virtual environments [MUST KNOW]

```
python3 -m venv venv                # create virtual env
source venv/bin/activate            # activate (Linux/Mac)
pip install boto3 requests pyyaml
pip freeze > requirements.txt        # snapshot dependencies
pip install -r requirements.txt      # install from file
deactivate
```

Why virtual environments matter: without one, `pip install` affects the system-wide Python, causing version conflicts across projects and potentially breaking system tools that depend on system Python (common issue on Debian/Ubuntu).

Quick Revision

- Always use a venv per project.
- `requirements.txt` pins dependencies for reproducible environments (CI/CD, teammates, prod).
- `__init__.py` marks a directory as a package (optional in modern Python but still common practice).

Must Remember

1. `python3 -m venv venv` then `source venv/bin/activate`.
2. Never `pip install` into system Python for project-specific packages.
3. `pip freeze > requirements.txt` before committing.
4. Pin versions in `requirements.txt` for production (`boto3==1.34.0`) to avoid surprise breakage.
5. Add `venv/` to `.gitignore`.

Practice

1. Create a venv, install `requests` and `boto3`, generate a `requirements.txt`.
2. Split a monitoring script into a `checks.py` module and a `main.py` that imports it.

---

11. Logging [MUST KNOW]

11.1 Why not just `print()`?

`print()` has no levels, no timestamps, no file output, and can't be turned on/off per module. `logging` is the production-grade standard.

11.2 Basic Setup

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[
        logging.StreamHandler(),                    # console
        logging.FileHandler("/var/log/myapp.log")    # file
    ]
)
logger = logging.getLogger(__name__)

logger.debug("Detailed diagnostic info")
logger.info("Deployment started")
logger.warning("Disk usage above 80%%")
logger.error("Failed to connect to database")
logger.critical("Service completely down")
logger.exception("Caught an error")   # use inside except: — logs full traceback
```

11.3 Levels (increasing severity)

Level	Numeric	Use for
DEBUG	10	verbose dev diagnostics
INFO	20	normal operation events
WARNING	30	something unexpected, not yet broken
ERROR	40	a failure occurred

Level	Numeric	Use for
CRITICAL	50	system-level failure

11.4 Rotating Logs [GOOD TO KNOW]

```
from logging.handlers import RotatingFileHandler

handler = RotatingFileHandler("/var/log/myapp.log", maxBytes=10_000_000, backupCount=5)
logger.addHandler(handler)
```

Prevents log files from growing unbounded — critical for long-running automation/daemons.

Quick Revision

- `logging` > `print()` for any real automation script.
- Set level appropriately: `DEBUG` for dev, `INFO`/`WARNING` for prod.
- `RotatingFileHandler` for long-lived scripts to avoid disk-filling logs.

Must Remember

1. `logging.basicConfig()` once, at the entry point of your script.
2. Use `logger = logging.getLogger(__name__)` per module, not the root logger directly.
3. `logger.exception()` inside `except:` blocks — captures the traceback automatically.
4. Never log secrets/passwords/tokens.
5. Rotate logs in any long-running/daemon-style script.

Practice

1. Add proper logging (console + file) to a disk-monitoring script.
2. Add a `RotatingFileHandler` capped at 5MB, 3 backups.
3. Replace all `print()` statements in an existing script with appropriate log levels.

---

12. Command-Line Arguments [MUST KNOW]

12.1 sys.argv (basic)

```
import sys
script_name = sys.argv[0]
args = sys.argv[1:]     # everything after the script name
```

Fine for trivial scripts; painful for anything with flags/options.

12.2 argparse (professional) [MUST KNOW]

```
import argparse

parser = argparse.ArgumentParser(description="Backup automation tool")
parser.add_argument("--source", required=True, help="Source directory")
```

```
parser.add_argument("--destination", required=True, help="Backup destination")
parser.add_argument("--dry-run", action="store_true", help="Simulate without copying")
parser.add_argument("--retries", type=int, default=3, help="Number of retry attempts")

args = parser.parse_args()

print(args.source, args.destination, args.dry_run, args.retries)
```

```
python backup.py --source /data --destination /backup --retries 5
python backup.py --source /data --destination /backup --dry-run
```

Quick Revision

- `argparse` gives you `--help`, type checking, required flags, and defaults for free.
- `action="store_true"` for boolean flags.

Must Remember

1. Always use `argparse` for any script that takes more than one raw positional argument.
2. `required=True` for mandatory flags, `default=` for optional ones.
3. `type=int`/`type=float` auto-converts and validates.
4. `--dry-run` flags are a DevOps best practice for destructive scripts.
5. `parser.add_argument("--help")` is automatic — don't add your own `-h`.

Practice

1. Build a CLI tool `disk_check.py --threshold 85 --path /`.
2. Add a `--verbose` flag that raises the logging level to DEBUG.
3. Add a subcommand-style tool: `deploy.py start|stop|status` using `add_subparsers()`.
   # 13. Working with APIs [MUST KNOW] (VERY IMPORTANT)

13.1 HTTP Basics

Method	Purpose
GET	Read data
POST	Create data
PUT	Replace/update data
PATCH	Partial update
DELETE	Remove data

Status code range	Meaning
2xx	Success
3xx	Redirect
4xx	Client error (bad request, auth, not found)
5xx	Server error

13.2 requests Library [MUST KNOW]

```python
import requests

resp = requests.get("https://api.example.com/instances", timeout=5)
resp.status_code
resp.json()
resp.headers

resp = requests.post(
    "https://api.example.com/deployments",
    json={"service": "web", "version": "1.4.2"},
    headers={"Authorization": f"Bearer {token}"},
    timeout=5
)
resp.raise_for_status()   # raises HTTPError on 4xx/5xx
```

13.3 Authentication

```python
# API key in header
headers = {"Authorization": f"Bearer {api_token}"}

# Basic auth
requests.get(url, auth=("user", "password"))
```

Never hardcode tokens — load from environment variables or a secrets manager (see Security section).

13.4 Error Handling, Timeouts, Retries [MUST KNOW]

```python
import requests
from requests.adapters import HTTPAdapter, Retry

session = requests.Session()
retries = Retry(total=3, backoff_factor=1, status_forcelist=[500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

try:
    resp = session.get("https://api.example.com/health", timeout=5)
    resp.raise_for_status()
except requests.exceptions.Timeout:
    logger.error("API request timed out")
except requests.exceptions.ConnectionError:
    logger.error("Could not connect to API")
except requests.exceptions.HTTPError as e:
    logger.error(f"API returned error: {e}")
```

Common mistake: not setting `timeout=` — a hung request can block your script forever.

Quick Revision

- `requests.get/post(...).json()` for JSON APIs.
- Always set `timeout=`, always call `raise_for_status()` or check `status_code`.

- Use `Retry`/`Session` for resilient automation against flaky APIs.

Must Remember

1. Always set a `timeout` on every request.
2. `raise_for_status()` to fail fast on 4xx/5xx.
3. Never hardcode API keys/tokens — use env vars.
4. Use a `Session` with retry/backoff for anything called repeatedly (health checks, polling).
5. Catch `requests.exceptions.*` specifically, not bare `except:`.

Practice

1. Write a function `check_api_health(url)` that returns True/False with proper timeout + error handling.
2. Add retry-with-backoff to a GET request against a flaky endpoint.
3. POST a JSON payload with a Bearer token and handle a 401 response gracefully.

---

14. AWS Automation with Python (Boto3) [MUST KNOW]

14.1 Setup

```
pip install boto3
```

```
import boto3

# Credentials resolved automatically from (in order):
# env vars -> ~/.aws/credentials -> IAM role (EC2/ECS instance profile) -> SSO
session = boto3.Session(profile_name="prod", region_name="us-east-1")
ec2 = session.client("ec2")            # low-level client (maps to API calls 1:1)
ec2_resource = session.resource("ec2")  # higher-level, object-oriented
```

On EC2/ECS in production: never hardcode keys — use an IAM instance/task role. `boto3` picks it up automatically.

14.2 EC2 Automation

```
import boto3

ec2 = boto3.client("ec2", region_name="us-east-1")

# List EC2 instances
resp = ec2.describe_instances()
for reservation in resp["Reservations"]:
    for instance in reservation["Instances"]:
        print(instance["InstanceId"], instance["State"]["Name"])

# Start / stop EC2
ec2.start_instances(InstanceIds=["i-0abc123"])
ec2.stop_instances(InstanceIds=["i-0abc123"])
```

```python
# Filter by tag
resp = ec2.describe_instances(
    Filters=[{"Name": "tag:Environment", "Values": ["prod"]}]
)
```

14.3 S3 Automation [MUST KNOW]

```python
s3 = boto3.client("s3")

# List buckets
[b["Name"] for b in s3.list_buckets()["Buckets"]]

# Upload a file
s3.upload_file("backup.tar.gz", "my-backup-bucket", "backups/backup.tar.gz")

# Download a file
s3.download_file("my-backup-bucket", "backups/backup.tar.gz", "restored.tar.gz")

# List objects in a bucket
resp = s3.list_objects_v2(Bucket="my-backup-bucket", Prefix="backups/")
for obj in resp.get("Contents", []):
    print(obj["Key"], obj["Size"])

# Delete an object
s3.delete_object(Bucket="my-backup-bucket", Key="old/backup.tar.gz")
```

14.4 IAM (read-heavy, careful with writes) [GOOD TO KNOW]

```python
iam = boto3.client("iam")
[u["UserName"] for u in iam.list_users()["Users"]]
iam.list_attached_user_policies(UserName="deploy-bot")
```

14.5 CloudWatch — Metrics & Alarms [GOOD TO KNOW]

```python
cw = boto3.client("cloudwatch")

resp = cw.get_metric_statistics(
    Namespace="AWS/EC2",
    MetricName="CPUUtilization",
    Dimensions=[{"Name": "InstanceId", "Value": "i-0abc123"}],
    StartTime=datetime.utcnow() - timedelta(hours=1),
    EndTime=datetime.utcnow(),
    Period=300,
    Statistics=["Average"]
)
```

14.6 Lambda [GOOD TO KNOW]

```
lam = boto3.client("lambda")
lam.invoke(FunctionName="my-function", InvocationType="Event", Payload=json.dumps({"key":
```

14.7 Auto Scaling & VPC (basics) [ADVANCED]

```
asg = boto3.client("autoscaling")
asg.describe_auto_scaling_groups()
asg.update_auto_scaling_group(AutoScalingGroupName="web-asg", DesiredCapacity=4)

vpc = boto3.client("ec2")
vpc.describe_vpcs()
vpc.describe_subnets(Filters=[{"Name": "vpc-id", "Values": ["vpc-123"]}])
```

14.8 Error Handling with Boto3

```
from botocore.exceptions import ClientError, NoCredentialsError

try:
    ec2.start_instances(InstanceIds=["i-0abc123"])
except ClientError as e:
    error_code = e.response["Error"]["Code"]
    if error_code == "InvalidInstanceID.NotFound":
        logger.error("Instance does not exist")
    else:
        logger.error(f"AWS error: {e}")
except NoCredentialsError:
    logger.error("AWS credentials not found")
```

Quick Revision

- `boto3.client()` = low-level 1:1 API mapping; `boto3.resource()` = higher-level OOP interface.
- Credential resolution order: env vars → `~/.aws/credentials` → IAM role → SSO.
- `ClientError` is the standard exception to catch for any AWS API call.

Must Remember

1. Never hardcode AWS access keys in scripts — use profiles/env vars/IAM roles.
2. `describe_instances()` response is deeply nested (`Reservations` → `Instances`) — a very common gotcha.
3. Use `Filters=[{"Name": ..., "Values": [...]}]` to avoid pulling entire account inventories.
4. Always wrap AWS calls in `try/except ClientError`.
5. Use IAM roles on EC2/ECS in production instead of static keys.

Practice

1. Write a script that lists all running EC2 instances with their tags.
2. Write a script that uploads all `.log` files in a directory to S3.

3. Write a script that stops all EC2 instances tagged `Environment=dev` outside business hours (cost optimization).

---

15. Automation Scripts [MUST KNOW]

Practical script skeletons you'll reuse constantly.

15.1 Backup Automation

```
import shutil, datetime
from pathlib import Path

def backup(source, dest_root):
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    dest = Path(dest_root) / f"backup_{timestamp}"
    shutil.copytree(source, dest)
    return dest
```

15.2 Log Cleanup

```
import time
from pathlib import Path

def cleanup_old_logs(log_dir, days=30):
    cutoff = time.time() - (days * 86400)
    for f in Path(log_dir).glob("*.log"):
        if f.stat().st_mtime < cutoff:
            f.unlink()
```

15.3 Disk / Service Monitoring

```
import shutil, subprocess

def check_disk(path="/", threshold=90):
    usage = shutil.disk_usage(path)
    percent_used = usage.used / usage.total * 100
    return percent_used < threshold

def check_service(name):
    result = subprocess.run(["systemctl", "is-active", name], capture_output=True, text=Tr
    return result.stdout.strip() == "active"
```

15.4 File Synchronization (simple)

```
import subprocess
```

```
def sync(source, dest):
    subprocess.run(["rsync", "-avz", "--delete", source, dest], check=True)
```

15.5 Health Checks

```
import requests

def health_check(url, expected_status=200):
    try:
        resp = requests.get(url, timeout=5)
        return resp.status_code == expected_status
    except requests.exceptions.RequestException:
        return False
```

15.6 Server Info Collector

```
import platform, psutil

def collect_server_info():
    return {
        "hostname": platform.node(),
        "os": platform.system(),
        "cpu_percent": psutil.cpu_percent(interval=1),
        "memory_percent": psutil.virtual_memory().percent,
        "disk_percent": psutil.disk_usage("/").percent,
    }
```

15.7 User Management (Linux)

```
import subprocess

def create_user(username):
    subprocess.run(["useradd", "-m", username], check=True)

def user_exists(username):
    result = subprocess.run(["id", username], capture_output=True)
    return result.returncode == 0
```

15.8 Deployment Automation (skeleton)

```
def deploy(version):
    logger.info(f"Deploying version {version}")
    subprocess.run(["git", "pull"], check=True)
    subprocess.run(["docker", "compose", "up", "-d", "--build"], check=True)
    if not health_check("http://localhost:8080/health"):
        raise DeploymentError("Health check failed after deployment")
    logger.info("Deployment successful")
```

15.9 AWS Resource Cleanup

```
def cleanup_unattached_volumes(ec2):
    volumes = ec2.describe_volumes(Filters=[{"Name": "status", "Values": ["available"]}])
    for vol in volumes["Volumes"]:
        ec2.delete_volume(VolumeId=vol["VolumeId"])
```

Quick Revision

- Every automation script follows the same shape: **check → act → verify → log**.
- Always separate "logic" functions from the "main" execution block (`if __name__ == "__main__":`).

Must Remember

1. `if __name__ == "__main__":` guards script entry points so functions are importable/testable.
2. Every destructive script should support `--dry-run`.
3. Log before and after every major action.
4. Verify success (health check) after any deployment/change.
5. Fail loud with proper exit codes — never fail silently.

Practice

1. Combine `check_disk` + `check_service` into a single health-report script with logging.
2. Write a log-cleanup script with a `--dry-run` flag.
3. Write a server-info collector that posts its JSON output to an API endpoint.

---

16. Python + Linux [MUST KNOW]

16.1 Processes & Signals

```
import os, signal

os.getpid()
os.kill(pid, signal.SIGTERM)   # graceful stop
os.kill(pid, signal.SIGKILL)   # force kill

def handle_sigterm(signum, frame):
    logger.info("Received SIGTERM, shutting down gracefully")
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)
```

16.2 Environment Variables, Files, Permissions

Covered in sections 6 & 7 — `os.environ`, `pathlib`, `os.chmod`.

16.3 Services (systemd) from Python

```
subprocess.run(["systemctl", "status", "nginx"])
subprocess.run(["systemctl", "restart", "nginx"])
subprocess.run(["journalctl", "-u", "nginx", "-n", "50"])
```

16.4 Cron

Python scripts are commonly triggered by cron rather than looping internally:

```
# crontab -e
*/5 * * * * /opt/venv/bin/python3 /opt/scripts/health_check.py >> /var/log/health_check.lo
```

Best practice: always use the venv's Python binary and redirect output to a log file in the crontab entry.

16.5 Running Shell Commands & Exit Codes

Already covered — `subprocess.run(..., check=True)` + `sys.exit(code)`.

Quick Revision

- Signal handling lets your daemon/long-running script shut down gracefully.
- Cron is the most common trigger for periodic Python automation — always log output and use the venv Python path.

Must Remember

1. Handle `SIGTERM` in any long-running script/daemon.
2. Cron entries: use full/absolute paths, and always redirect stdout/stderr.
3. `journalctl -u <service> -n 50` — quick way to pull recent service logs from Python.
4. systemd status/restart is almost always driven via `subprocess`, not a native Python systemd API, unless using `python-systemd`.

Practice

1. Add a `SIGTERM` handler to a long-running polling script.
2. Write a cron-ready health-check script with absolute paths and log redirection.
3. Pull the last 20 lines of a service's journal log from Python and check for `"ERROR"`.

---

17. Python Best Practices [MUST KNOW]

- **PEP 8**: 4-space indents, `snake_case` names, max ~79-99 char lines, blank lines between functions.
- **Meaningful names**: `check_disk_usage()` not `chk()`.
- **Small, single-purpose functions** — easier to test and reuse.
- **Modular code**: split scripts into `utils/`, `config.py`, `main.py` once they grow past ~150 lines.
- **Consistent error handling & logging** (sections 5 & 11).
- **Configuration management**: externalize thresholds/hostnames/paths into a `config.yml` or env vars — never hardcode.

- **Secrets management**: use environment variables, `.env` files (not committed), or a secrets manager (AWS Secrets Manager / SSM Parameter Store / Vault). Never commit secrets to git.
- **`.env` files** with `python-dotenv`:

```
from dotenv import load_dotenv
import os
load_dotenv()   # loads .env into os.environ
db_password = os.environ["DB_PASSWORD"]
```

- **`requirements.txt`** pinned for prod, virtual environments always.
- **Reusable scripts**: wrap logic in functions callable both as a CLI and importable module.

Quick Revision

- Code that looks like it was written by a team, not a one-off hack: consistent style, small functions, externalized config, no hardcoded secrets.

Must Remember

1. PEP 8 formatting (or use `black`/`ruff` to auto-format).
2. No hardcoded credentials, ever — `.env` + `python-dotenv` or a secrets manager.
3. Config in files/env vars, not inline constants scattered through code.
4. `requirements.txt` pinned for reproducibility.
5. Structure: `main.py` (entry) + `utils/` (logic) once a script grows.

Practice

1. Refactor a 100-line "everything in one function" script into 4-5 smaller functions.
2. Move all hardcoded values (thresholds, paths) into a `config.yml`.
3. Set up `.env` + `python-dotenv` for a script that currently hardcodes an API key.

---

18. Testing Basics [GOOD TO KNOW]

18.1 unittest

```
import unittest

def add_tag_prefix(tag):
    return f"env-{tag}"

class TestTagging(unittest.TestCase):
    def test_prefix_added(self):
        self.assertEqual(add_tag_prefix("prod"), "env-prod")

if __name__ == "__main__":
    unittest.main()
```

18.2 pytest (preferred in industry) [MUST KNOW]

```
pip install pytest
```

```
# test_utils.py
from utils import check_disk

def test_check_disk_under_threshold():
    assert check_disk("/", threshold=99) is True
```

```
pytest -v
```

18.3 Mocking (avoid hitting real AWS/APIs in tests) [GOOD TO KNOW]

```
from unittest.mock import patch

@patch("boto3.client")
def test_list_instances(mock_client):
    mock_client.return_value.describe_instances.return_value = {"Reservations": []}
    result = list_instances()
    assert result == []
```

Quick Revision

- `pytest` is the industry default; `unittest` is stdlib and fine for simple cases.
- Mock external calls (AWS, HTTP) in unit tests — never hit real infrastructure in a test suite.

Must Remember

1. `pytest` auto-discovers `test_*.py` files and `test_*` functions.
2. `assert` statements, not `self.assertEqual` (that's `unittest`-specific).
3. Mock `boto3`/`requests` calls — tests should be fast and offline.
4. Test the logic (parsing, thresholds, decisions), not the AWS API itself.

Practice

1. Write a pytest test for a `parse_log_line()` function.
2. Mock a `requests.get` call in a test for `health_check()`.
3. Write a test that checks a threshold function returns the correct alert level.

---

19. Python Security Basics [MUST KNOW]

1. **Never hardcode passwords/API keys/AWS keys** in source code — use env vars or a secrets manager.
2. **Environment variables** for local secrets; never commit `.env` to git (`.gitignore` it).
3. **Secret managers**: AWS Secrets Manager, SSM Parameter Store (SecureString), HashiCorp Vault — for production-grade secret storage/rotation.
4. **Input validation**: never trust external input (API payloads, CLI args, file contents) — validate types/ranges before use.

5. **Safe subprocess usage**: avoid `shell=True`; pass argument lists; never string-format user input into a shell command.
6. **Avoiding shell injection**:

```
# DANGEROUS
os.system(f"ping {user_input}")

# SAFE
subprocess.run(["ping", "-c", "1", user_input])
```

7. **File permission considerations**: secrets files should be `chmod 600`; scripts writing sensitive files should set restrictive permissions explicitly.

Quick Revision

- Treat every external input (CLI arg, env var from an untrusted source, API payload) as potentially hostile.
- Secrets belong in a secrets manager or environment, never in source code or git history.

Must Remember

1. No hardcoded secrets — ever, not even "temporarily".
2. `shell=True` is a red flag — justify it or avoid it.
3. `.gitignore` your `.env` and any credentials files.
4. Validate/sanitize all external input before using it in commands or queries.
5. `chmod 600` for files containing secrets.

Practice

1. Refactor a script that hardcodes an AWS key into using env vars.
2. Rewrite an `os.system(f"...")` call using safe `subprocess.run([...])`.
3. Add input validation to a CLI tool that accepts a hostname/IP argument.

---

20. Python Interview Revision [MUST KNOW]

Common Python Questions

- Difference between list, tuple, set, dict — and when to use each.
- Mutable vs immutable types (list/dict/set are mutable; str/tuple/int are immutable).
- `is` vs `==`.
- What happens with a mutable default argument (`def f(x=[])`)? — classic gotcha.
- Shallow vs deep copy (`copy.copy` vs `copy.deepcopy`).
- What is a decorator? (a function that wraps another function to add behavior).
- Generator vs list (`yield` — lazy, memory-efficient iteration).
- GIL (Global Interpreter Lock) — why Python threads don't give true CPU parallelism; use `multiprocessing` for CPU-bound work.
- `*args`/`**kwargs` — explain and give an example.
- Context managers (`with`) — what problem do they solve?

Common Mistakes to Avoid Mentioning You've Made (but know them!)

- Bare `except:` blocks.

- Mutable default arguments.
- Not closing files (not using `with`).
- `shell=True` with unsanitized input.
- Comparing floats with == instead of `math.isclose()`.

Frequently Used Methods (rapid recall)

`str: strip, split, join, replace, format, startswith, endswith list: append, extend, sort, pop, remove, index dict: get, keys, values, items, update, pop`

DevOps Python Interview Questions

- How would you check if a Linux service is running from Python?
- How do you securely handle AWS credentials in a script?
- How would you write a script to clean up old log files safely?
- Difference between `subprocess.run` and `subprocess.Popen`?
- How do you parse and monitor a log file for error patterns?
- How would you structure a CLI automation tool?

Boto3 Interview Questions

- How does boto3 resolve AWS credentials?
- Difference between `client` and `resource` in boto3?
- How do you handle pagination in boto3 (`describe_instances` can return truncated results — use paginators)?

```
paginator = ec2.get_paginator("describe_instances")
for page in paginator.paginate():
    ...
```

- How do you catch and handle a specific AWS error (e.g., instance not found)?
- How would you write a script to stop all dev EC2 instances outside business hours?

Automation-Related Questions

- How do you make a script idempotent (safe to run multiple times)?
- How do you handle retries for a flaky operation?
- How would you add a `--dry-run` flag to a destructive script?
- How do you ensure a cron-triggered script's failures are visible (logging + exit codes + alerting)?

Quick Revision

- Be ready to explain **why**, not just **what** — e.g., not just "use subprocess" but "avoid shell=True because of injection risk."

Must Remember

1. Mutable default argument gotcha — always mention `None` + assign inside function as the fix.
2. GIL → use `multiprocessing` for CPU-bound, `threading`/`asyncio` for I/O-bound.
3. boto3 credential resolution order.
4. Idempotency and `--dry-run` as automation best practices.

5. Always tie a Python concept back to a concrete DevOps scenario in interviews.

Practice

1. Explain in your own words: what problem does `with open(...)` solve?
2. Write and explain a boto3 paginator example.
3. Explain the mutable default argument bug with a live code example.

PART 2 — BASIC BASH SCRIPTING

1. Linux Shell Basics [MUST KNOW]

- **Shell**: a program that interprets commands you type and passes them to the OS.
- **Bash**: the most common Linux shell (Bourne Again SHell) — what almost all DevOps scripting targets.
- **Terminal**: the program hosting the shell session.
- **Command execution**: `command [options] [arguments]`, e.g. `ls -la /var/log`.
- **PATH**: an environment variable listing directories searched for executables.

```
echo $PATH
which python3        # shows the resolved path of a command
```

- **Environment variables**: `export VAR=value` — inherited by child processes.

```
export AWS_REGION=us-east-1
env | grep AWS
```

Quick Revision

- Bash is the shell you'll script in for 95% of Linux DevOps work.
- $PATH determines which binary runs when you type a command name.

Must Remember

1. `which <cmd>` to find what will actually execute.
2. `export` makes a variable visible to child processes.
3. env lists all current environment variables.

Practice

1. Print your $PATH and identify how many directories it contains.
2. `export` a variable and confirm a subshell (`bash -c 'echo $VAR'`) can see it.

---

2. Bash Script Basics [MUST KNOW]

```
#!/bin/bash
echo "Hello DevOps"
```

- **Shebang** (`#!/bin/bash`): tells the OS which interpreter runs the file. Always the first line.
- **Make executable**: `chmod +x script.sh`
- **Run it**: `./script.sh` (needs execute permission + shebang) or `bash script.sh` (doesn't need either).
- **Comments**: `# this is a comment`

Common mistake: forgetting `chmod +x` then getting `Permission denied` — or forgetting the shebang and the script running under the wrong shell (`sh` vs `bash`, which differ subtly).

Quick Revision

- Shebang + `chmod +x` = the standard way every Bash script starts.

Must Remember

1. `#!/bin/bash` as line 1, always.
2. `chmod +x script.sh` before `./script.sh`.
3. `bash script.sh` runs regardless of permissions.

Practice

1. Write and run a "Hello DevOps" script both ways (`./` and `bash`).
2. Deliberately omit the shebang, run it with `sh`, and observe any difference.

---

3. Variables [MUST KNOW]

```
name="web01"
count=5
echo "Server: $name"
echo "Server: ${name}"          # braces avoid ambiguity, e.g. ${name}01
```

- **No spaces around =** — `name = "web01"` is a syntax error in Bash (looks like a command called name).
- **Command substitution**:

```
today=$(date +%F)
files=$(ls /var/log)
```

- **Environment variables**: `$HOME`, `$USER`, `$PATH`, `$PWD`.
- **Arithmetic**:

```
count=$((count + 1))
```

Quick Revision

- `$(command)` is the modern, preferred substitution syntax (backticks are legacy).
- `${var}` when you need to disambiguate from surrounding text.

Must Remember

1. No spaces around = in assignment.
2. `$(...)` not backticks for command substitution.
3. `$((...))` for arithmetic.
4. Always quote variable expansions: `"$name"` — unquoted variables break on spaces/globs.

Practice

1. Store `hostname` command output in a variable and print it.
2. Increment a counter variable in a loop using `$(( ))`.

---

4. User Input [MUST KNOW]

```
read -p "Enter environment: " env
echo "You entered: $env"
```

Script Arguments

```
#!/bin/bash
echo "Script name: $0"
echo "First arg: $1"
echo "Second arg: $2"
echo "All args: $@"
echo "Number of args: $#"
echo "Exit code of last command: $?"
```

```
./deploy.sh production v1.4.2
# $0 = ./deploy.sh, $1 = production, $2 = v1.4.2, $# = 2
```

Common mistake: using `$*` when you meant `"$@"` — `"$@"` preserves each argument as a separate word (important when args contain spaces); `$*` joins them into one string.

Quick Revision

- Positional params `$1`, `$2`... ; `$#` count; `$@` all args; `$?` last exit code — these four are used constantly.

Must Remember

1. `$?` immediately after a command to check success (`0`) or failure (non-zero).
2. `"$@"` (quoted) to safely pass all arguments through to another command.
3. `read -p "prompt: " var` for interactive scripts.
4. Always validate `$#` before using `$1`/`$2` (avoid “unbound variable” errors).

Practice

1. Write a script requiring exactly 2 arguments; print usage and exit 1 if not provided.
2. Check `$?` after a deliberately failing command (e.g. `ls /nonexistent`).
3. Write a script using `read` to confirm a destructive action (`y/n`).

---

5. Conditions [MUST KNOW]

```
if [ "$env" == "production" ]; then
    echo "Deploying to prod"
elif [ "$env" == "staging" ]; then
    echo "Deploying to staging"
else
    echo "Unknown environment"
fi
```

Test Operators

Test	Meaning
`-z "$s"`	string is empty
`-n "$s"`	string is not empty
`"$a" == "$b"`	string equal
`"$a" != "$b"`	string not equal
`-eq -ne -gt -lt -ge -le`	numeric comparisons
`-f file`	file exists and is a regular file
`-d dir`	directory exists
`-e path`	path exists (any type)
`-x file`	file is executable
`-r / -w`	readable / writable

```
if [ -f "/etc/nginx/nginx.conf" ]; then
    echo "Config exists"
fi

if [ "$count" -gt 90 ]; then
    echo "Threshold exceeded"
fi
```

[ ] vs [[ ]] [GOOD TO KNOW]

`[[ ]]` is a Bash-specific improvement: supports &&/|| directly, pattern matching, and doesn't word-split unquoted variables the same dangerous way.

```
if [[ "$env" == "prod" || "$env" == "production" ]]; then
    echo "Production deploy"
fi
```

Common mistake: `[ $count -gt 90 ]` without quotes — if `$count` is empty/unset, this becomes `[ -gt 90 ]`, a syntax error. Always quote: `[ "$count" -gt 90 ]`.

Quick Revision

- `[[ ]]` (Bash) is safer/more featureful than POSIX `[ ]` — prefer it in Bash-only scripts.
- File-test operators (`-f`, `-d`, `-e`) are used constantly for pre-flight checks.

Must Remember

1. Always quote variables inside `[ ]`/`[[ ]]`.
2. `-eq`/`-ne`/`-gt`/`-lt` for numbers, ==/!= for strings — don't mix them up.
3. `[[ ]]` supports &&/|| inline; `[ ]` needs `-a`/`-o` (avoid those, use separate `[[ ]] && [[ ]]` instead).
4. `-f` vs `-d` vs `-e` — file vs directory vs "exists at all".

Practice

1. Write a script checking if `/etc/myapp/config.yml` exists before proceeding.
2. Write a threshold check: if disk usage (a variable) `-ge 90`, print CRITICAL.
3. Convert a `[ ]`-based script to use `[[ ]]` with `||`.

6. Loops [MUST KNOW]

```bash
# for loop over a list
for server in web01 web02 db01; do
    echo "Checking $server"
done

# for loop over command output
for file in $(ls /var/log/*.log); do
    echo "$file"
done

# C-style for loop
for (( i=0; i<5; i++ )); do
    echo "$i"
done

# while loop
count=0
while [ "$count" -lt 5 ]; do
    echo "$count"
    count=$((count + 1))
done

# until loop (opposite of while)
until [ -f /tmp/ready ]; do
    echo "Waiting..."
    sleep 2
done

# break / continue
for server in web01 web02 db01; do
    if [ "$server" == "db01" ]; then
        continue
    fi
    echo "$server"
done
```

Common mistake: `for file in $(ls *.log)` breaks on filenames with spaces — prefer `for file in *.log` (globbing) when possible.

Quick Revision

- `for...in` for iterating known lists; `while/until` for condition-driven loops (retries, polling).
- Prefer glob (`*.log`) over `$(ls ...)` for iterating files.

Must Remember

1. `while [ condition ]; do ... done` — condition checked before each iteration.

2. `until` = “loop while false” — good for “wait until X exists” patterns.
3. `break`/`continue` work the same as in Python.
4. Glob patterns over parsing `ls` output for file iteration.

Practice

1. Write a `for` loop that pings a list of 3 hosts.
2. Write a `while` loop that polls for a file’s existence, sleeping 5s between checks, max 10 tries.
3. Write a C-style `for` loop counting down from 10 to 1.

---

7. Functions [MUST KNOW]

```bash
check_disk() {
    local mount=$1
    local threshold=${2:-90}     # default value if $2 not provided
    usage=$(df --output=pcent "$mount" | tail -1 | tr -d '% ')
    if [ "$usage" -ge "$threshold" ]; then
        echo "CRITICAL: $mount at ${usage}%"
        return 1
    fi
    echo "OK: $mount at ${usage}%"
    return 0
}

check_disk "/" 85
echo "Exit code: $?"
```

- **`local`**: scopes a variable to the function — always use it to avoid polluting global scope.
- **Return codes**: `return N` (0-255) — `return` is for status, not data. To “return” data, echo it and capture with `$(function_name)`.

```bash
get_hostname() {
    echo "$(hostname)"
}
host=$(get_hostname)
```

Quick Revision

- Bash functions “return” via exit code (0-255) OR by echoing output captured with `$()`.
- `local` keeps function variables from leaking into global scope.

Must Remember

1. `local var=value` inside every function.
2. `return` = exit status (0=success), not a data return like Python.
3. Use `echo`+`$(func)` to get “data” out of a function.
4. Default argument pattern: `${2:-default}`.

Practice

1. Write a `service_check()` function returning 0/1 based on `systemctl is-active`.
2. Write a function that echoes the current disk usage percentage for a given mount, called via `$()`.
3. Add a default threshold parameter to a disk-check function. # 8. Important Linux Commands for Bash Automation [MUST KNOW]

For each command: what it does, basic syntax, key examples, DevOps use case.

Navigation & File Basics

`ls` — list directory contents. `ls -la /var/log` (long format + hidden files), `ls -lh` (human-readable sizes), `ls -lt` (sort by modified time). *Use case:* quick inventory of log/config directories.

`cd` — change directory. `cd /var/log`, `cd -` (previous dir), `cd ~` (home).

`pwd` — print working directory. Useful in scripts to confirm/log current location.

`mkdir` — make directory. `mkdir -p /opt/app/{logs,config,data}` (creates nested dirs + brace expansion, no error if exists).

`touch` — create empty file / update timestamp. `touch app.log`, `touch -d "2 days ago" file` (backdate for testing).

`cp` — copy. `cp -r source/ dest/` (recursive), `cp -a` (archive: preserves permissions/timestamps, best for backups).

`mv` — move/rename. `mv old.conf new.conf`, `mv *.log /archive/`.

`rm` — remove. `rm file.txt`, `rm -rf /tmp/build/` (recursive, force — **dangerous**, always double-check the path in automation).

Viewing Files

`cat` — print whole file / concatenate files. `cat access.log`, `cat file1 file2 > combined.log`.

`less` — page through large files interactively (`/` to search, `q` to quit). Preferred over `cat` for large logs.

`head` — first N lines. `head -n 50 app.log`.

`tail` — last N lines. `tail -n 100 app.log`, `tail -f app.log` (follow — live log tailing, extremely common).

Search & Text Processing [MUST KNOW]

`grep` — search text by pattern. `grep "ERROR" app.log`, `grep -i "error"` (case-insensitive), `grep -r "TODO" ./src` (recursive), `grep -v "DEBUG"` (invert match), `grep -c "ERROR"` (count matches), `grep -E "ERROR|WARN"` (extended regex). *Use case:* the single most-used command for log analysis and CI/CD output filtering.

`find` — locate files by criteria. `find /var/log -name "*.log"`, `find /tmp -mtime +7 -delete` (files older than 7 days, deleted), `find / -size +100M`, `find . -type d -name "node_modules"`. *Use case:* automated cleanup jobs, locating config files across a filesystem.

`awk` — pattern-based text processing, especially column extraction. `awk '{print $1}' access.log` (print 1st column), `df -h | awk '{print $5, $6}'` (usage % + mount), awk

`-F: '{print $1}' /etc/passwd` (custom delimiter). *Use case:* pulling specific fields out of structured command output.

sed — stream editor, find & replace. `sed 's/old/new/g' file.txt`, `sed -i 's/DEBUG/INFO/g' config.yml` (in-place edit), `sed -n '5,10p' file` (print lines 5-10). *Use case:* bulk config file edits in deployment scripts.

sort — sort lines. `sort file.txt`, `sort -n` (numeric), `sort -r` (reverse), `sort -k2` (sort by 2nd column).

uniq — remove/report duplicate adjacent lines (use with `sort` first). `sort access.log | uniq -c | sort -rn` (frequency count, very common pattern).

cut — extract columns/fields. `cut -d',' -f1,3 data.csv` (fields 1 and 3, comma-delimited).

xargs — build/execute commands from stdin. `find . -name "*.log" | xargs rm`, `cat hosts.txt | xargs -I{} ssh {} uptime`. *Use case:* applying a command to a list of items produced by another command.

wc — word/line/byte count. `wc -l file.txt` (line count — very common for counting log entries).

tr — translate/delete characters. `tr 'a-z' 'A-Z' < file`, `tr -d ' '` (strip spaces).

tee — write output to a file AND stdout simultaneously. `echo "test" | tee -a log.txt`. *Use case:* logging a command's output while still seeing it live in the terminal.

Permissions & Ownership

chmod — change permissions. `chmod +x deploy.sh`, `chmod 644 config.yml`, `chmod -R 755 /opt/app`.

chown — change owner. `chown appuser:appgroup /opt/app`, `chown -R www-data:www-data /var/www`.

Process & Resource Monitoring [MUST KNOW]

ps — snapshot of running processes. `ps aux`, `ps aux | grep nginx`, `ps -ef --forest` (tree view).

top / **htop** — live process/resource monitor. `top -bn1` (batch mode, one snapshot — scriptable).

df — disk space usage. `df -h` (human-readable), `df -h /` (specific mount).

du — directory/file space usage. `du -sh /var/log` (summary, human-readable), `du -sh /var/log/* | sort -rh` (largest subdirs first).

free — memory usage. `free -m` (MB), `free -h`.

uptime — system uptime + load average. Used constantly in health-check scripts.

Services & Logs (systemd)

systemctl — manage services. `systemctl status nginx`, `systemctl restart nginx`, `systemctl is-active nginx`, `systemctl enable nginx` (start on boot), `systemctl list-units --type=service --state=failed`.

journalctl — view systemd logs. `journalctl -u nginx -n 50` (last 50 lines for a unit), `journalctl -u nginx -f` (follow), `journalctl --since "1 hour ago"`.

Network & Transfer

curl — transfer data / test HTTP endpoints. `curl -I https://example.com` (headers only), `curl -s -o /dev/null -w "%{http_code}" URL` (just the status code — great for health checks), `curl -X POST -d '{"k":"v"}' -H "Content-Type: application/json" URL`.

wget — download files. `wget -q URL -O output.file`.

tar — archive files. `tar -czvf backup.tar.gz /data` (create, gzip), `tar -xzvf backup.tar.gz` (extract), `tar -tzvf backup.tar.gz` (list contents without extracting).

ssh — remote shell. `ssh user@host`, `ssh -i key.pem user@host "systemctl status nginx"` (run remote command).

scp — secure copy over SSH. `scp file.txt user@host:/remote/path/`, `scp -r dir/ user@host:/remote/path/`.

rsync — efficient sync/copy (only transfers changes). `rsync -avz src/ user@host:/dest/`, `rsync -avz --delete src/ dest/` (mirror, deletes extras at destination). *Use case:* backups and deployments — far more efficient than `scp` for repeated syncs.

Quick Revision

- `grep`, `awk`, `sed`, `find`, `tail -f` are the everyday log/text toolkit.
- `df`, `du`, `free`, `top -bn1`, `uptime` are the everyday resource-check toolkit.
- `systemctl` + `journalctl` for service management/troubleshooting.
- `curl`, `rsync`, `scp`, `ssh` for network/remote operations.

Must Remember

1. `grep -r`, `awk '{print $N}'`, `sed -i 's/x/y/g'` — memorize these three cold.
2. `tail -f` for live log watching; `journalctl -u <svc> -f` for live service logs.
3. `find ... -mtime +N -delete` for automated cleanup (test with `-print` first, not `-delete`, to avoid disasters).
4. `rsync -avz --delete` for mirroring; `--delete` is powerful and dangerous — dry-run with `--dry-run` first.
5. `curl -s -o /dev/null -w "%{http_code}"` is the standard one-liner for scripted health checks.

Practice

1. Use `grep` + `wc -l` to count ERROR lines in a log file.
2. Use `awk` to extract the 5th column (usage %) from `df -h` output.
3. Use `find` + `-mtime +7` to list (not delete) log files older than a week.
4. Write a `curl` one-liner that prints only the HTTP status code of a URL.

---

9. Pipes and Redirection [MUST KNOW]

Operator	Meaning
|	pipe stdout of one command into stdin of the next
>	redirect stdout, **overwrite** file

Operator	Meaning
>>	redirect stdout, **append** to file
<	redirect stdin from a file
2>	redirect stderr only
2>&1	redirect stderr to wherever stdout is currently going
&	run command in background
&&	run next command only if previous **succeeded** (exit code 0)
||	run next command only if previous **failed** (non-zero exit code)

```
# Pipe: count ERROR lines
grep "ERROR" app.log | wc -l

# Redirect stdout, overwrite
df -h > disk_report.txt

# Redirect stdout, append
echo "Deploy finished at $(date)" >> deploy.log

# Redirect stderr only
./script.sh 2> errors.log

# Redirect both stdout and stderr to the same file (very common in cron)
./script.sh > output.log 2>&1

# Background execution
long_running_task.sh &

# Chain: only restart if build succeeds
./build.sh && systemctl restart myapp

# Chain: fallback command if primary fails
curl -sf http://primary/health || curl -sf http://backup/health
```

Common mistake: `./script.sh 2>&1 > output.log` — order matters! This sends stderr to the *old* stdout (terminal), then redirects stdout to the file. Correct order: `> output.log 2>&1`.

Quick Revision

- 2>&1 must come **after** `> file` to redirect both streams to the same file.
- &&/|| build simple conditional pipelines without full `if` statements.

Must Remember

1. `> output.log 2>&1` — the standard cron-job redirection pattern.
2. \| chains commands; &&/\|\| chain based on success/failure.
3. >> append vs > overwrite — a very easy mistake to make with logs.
4. & backgrounds a process; combine with `wait` to block until it finishes.

Practice

1. Run a build script, and only deploy if it succeeds, using &&.
2. Redirect both stdout and stderr of a script to a single log file correctly.
3. Write a fallback health-check using ||.

---

10. Bash + DevOps [MUST KNOW]

Disk Usage Monitoring

```
#!/bin/bash
THRESHOLD=90
df -h --output=pcent,target | tail -n +2 | while read -r pcent mount; do
    pcent=${pcent//%/}
    if [ "$pcent" -ge "$THRESHOLD" ]; then
        echo "CRITICAL: $mount at ${pcent}%"
    fi
done
```

CPU / Memory Monitoring

```
#!/bin/bash
cpu_idle=$(top -bn1 | grep "Cpu(s)" | awk '{print $8}')
cpu_used=$(echo "100 - $cpu_idle" | bc)
echo "CPU usage: ${cpu_used}%"

mem_used_pct=$(free | awk '/Mem:/ {printf "%.0f", $3/$2 * 100}')
echo "Memory usage: ${mem_used_pct}%"
```

Backup

```
#!/bin/bash
SRC="/data"
DEST="/backup/data_$(date +%Y%m%d_%H%M%S).tar.gz"
tar -czf "$DEST" "$SRC" && echo "Backup created: $DEST" || echo "Backup FAILED"
```

Log Cleanup

```
#!/bin/bash
find /var/log/myapp -name "*.log" -mtime +30 -exec rm {} \;
echo "Old logs cleaned up on $(date)"
```

Service Health Check + Auto-Restart

```
#!/bin/bash
SERVICE="nginx"
if ! systemctl is-active --quiet "$SERVICE"; then
```

```
    echo "$SERVICE is down, restarting..."
    systemctl restart "$SERVICE"
    sleep 2
    if systemctl is-active --quiet "$SERVICE"; then
        echo "$SERVICE restarted successfully"
    else
        echo "$SERVICE FAILED to restart" >&2
        exit 1
    fi
fi
```

Check if a Process is Running

```
if pgrep -x "nginx" > /dev/null; then
    echo "nginx is running"
else
    echo "nginx is NOT running"
fi
```

Check if a Website is Responding

```
#!/bin/bash
URL="https://example.com"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$URL")
if [ "$STATUS" -eq 200 ]; then
    echo "$URL is UP ($STATUS)"
else
    echo "$URL is DOWN (status: $STATUS)"
fi
```

Simple Deployment Script

```
#!/bin/bash
set -euo pipefail   # exit on error, undefined var, or pipe failure — see below

echo "Pulling latest code..."
git pull origin main

echo "Rebuilding containers..."
docker compose up -d --build

echo "Waiting for health check..."
sleep 5
STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/health)
if [ "$STATUS" -ne 200 ]; then
    echo "Deployment health check failed!" >&2
    exit 1
fi
echo "Deployment successful."
```

`set -euo pipefail` [MUST KNOW] — put this near the top of every serious Bash script: `- -e:`

exit immediately if any command fails. - `-u`: treat unset variables as an error (catches typos in variable names). - `-o pipefail`: a pipeline fails if *any* command in it fails, not just the last one.

Docker Cleanup

```
#!/bin/bash
docker container prune -f
docker image prune -af
docker volume prune -f
docker system df   # report space usage after cleanup
```

Log Collection

```
#!/bin/bash
OUTDIR="/tmp/log_bundle_$(date +%Y%m%d)"
mkdir -p "$OUTDIR"
journalctl -u nginx -n 500 > "$OUTDIR/nginx.log"
cp /var/log/syslog "$OUTDIR/" 2>/dev/null || true
tar -czf "${OUTDIR}.tar.gz" "$OUTDIR"
echo "Logs bundled at ${OUTDIR}.tar.gz"
```

Quick Revision

- `set -euo pipefail` should be near-default at the top of production Bash scripts.
- `systemctl is-active --quiet` + `pgrep` are the standard building blocks for health checks.
- Combine `curl -w "%{http_code}"` with an `if` for scripted uptime checks.

Must Remember

1. `set -euo pipefail` for any script that isn't a trivial one-liner.
2. `systemctl is-active --quiet <svc>` returns exit code 0/1 — perfect for `if`.
3. `pgrep -x <name>` to check if a process is running by exact name.
4. Always redirect deployment/cron script output to a log file.
5. Docker cleanup (`prune`) commands are a common cost/disk-space task.

Practice

1. Write a disk-monitoring script that alerts (prints) any mount over 85%.
2. Write a service auto-restart script for a service of your choice, with logging.
3. Add `set -euo pipefail` to an existing script and fix whatever it breaks (usually reveals unset variables).

PART 3 — PYTHON VS BASH [MUST KNOW]

When to use Bash

- Quick, linear sequences of existing Linux commands (glue code).
- Simple file/service checks that fit in <30 lines.
- Anything that's 90% calling other CLI tools with minimal logic.
- Cron jobs / systemd `ExecStart` one-liners.
- CI/CD pipeline steps (most CI YAML `run:` blocks are Bash).

When to use Python

- Anything with real logic: branching, data transformation, retries, structured error handling.
- Working with JSON/YAML/APIs.
- AWS/cloud SDK automation (`boto3` has no clean Bash equivalent).
- Code that needs to be tested (`pytest`), reused as a module, or maintained by a team long-term.
- Anything processing more than trivial data structures (nested dicts/lists).

Comparison Table

Task	Bash	Python
Linux commands	Native, first-class	Via `subprocess` (extra layer)
File manipulation	Fine for simple ops	Better for complex logic (renaming rules, conditional processing)
APIs	Awkward (`curl` + `jq` parsing)	Native (`requests`, `.json()`)
AWS automation	AWS CLI works, but limited logic	`boto3` — the standard for real automation
JSON	Painful (needs `jq`)	Native (`json` module)
Large automation projects	Becomes unmaintainable past ~100 lines	Scales well (modules, tests, types)
System administration	Excellent — this is Bash's home turf	Very good, slightly more verbose
CI/CD	Default language of pipeline steps	Used for the pipeline's actual logic/tooling
Log processing	`grep`/`awk`/`sed` are extremely fast for simple filters	Better for structured parsing + aggregation + alerting logic

Same Task, Both Languages

Task: Alert if disk usage on / exceeds 90%

```bash
# Bash
usage=$(df --output=pcent / | tail -1 | tr -d '% ')
if [ "$usage" -ge 90 ]; then
    echo "CRITICAL: disk at ${usage}%"
fi
```

```python
# Python
import shutil
```

```python
usage = shutil.disk_usage("/")
percent = usage.used / usage.total * 100
if percent >= 90:
    print(f"CRITICAL: disk at {percent:.0f}%")
```

Task: List all S3 buckets

```bash
# Bash (AWS CLI)
aws s3api list-buckets --query "Buckets[].Name" --output text
```

```python
# Python (boto3)
import boto3
s3 = boto3.client("s3")
print([b["Name"] for b in s3.list_buckets()["Buckets"]])
```

Task: Count ERROR lines in a log

```bash
# Bash
grep -c "ERROR" app.log
```

```python
# Python
count = sum(1 for line in open("app.log") if "ERROR" in line)
```

Quick Revision

- Bash = glue and CLI-native tasks. Python = logic, data, APIs, cloud SDKs, anything that needs to scale or be tested.
- Real DevOps engineers use **both** — Bash inside CI YAML/cron, Python for the substantial automation.

Must Remember

1. Don't write a 200-line Bash script with nested `if`s and JSON parsing via `jq` — that's a Python job.
2. Don't wrap a single `systemctl restart` command in a 30-line Python script — that's a Bash job.
3. `boto3` » AWS CLI-in-Bash for anything beyond a one-off command.
4. JSON = Python's home turf; simple text/log filtering = Bash's home turf.

Practice

1. Implement "check if a service is running" in both Bash and Python — compare line count and readability.
2. Implement "upload a file to S3" in both AWS CLI (Bash) and boto3 (Python).
3. Decide: would you write a multi-step deployment orchestrator (build → test → deploy → verify → rollback) in Bash or Python? Justify it.

PART 4 — DEVOPS SCRIPTING PATTERNS [MUST KNOW]

Reusable snippets you'll rewrite in nearly every automation script — memorize the shape, not just the code.

Check if a Command Exists

```bash
# Bash
if ! command -v docker &> /dev/null; then
    echo "docker is not installed" >&2
    exit 1
fi
```

```python
# Python
import shutil
if shutil.which("docker") is None:
    raise EnvironmentError("docker is not installed")
```

Check if a File Exists

```bash
[ -f "/etc/myapp/config.yml" ] || { echo "Config missing"; exit 1; }
```

```python
from pathlib import Path
if not Path("/etc/myapp/config.yml").exists():
    raise FileNotFoundError("Config missing")
```

Check if a Service is Running

```bash
systemctl is-active --quiet nginx && echo "up" || echo "down"
```

```python
import subprocess
def is_active(service):
    r = subprocess.run(["systemctl", "is-active", "--quiet", service])
    return r.returncode == 0
```

Retry an Operation

```bash
for i in 1 2 3; do
    curl -sf http://api.example.com/health && break
    echo "Attempt $i failed, retrying..."
    sleep 2
done
```

```python
import time
def retry(func, attempts=3, delay=2):
    for i in range(1, attempts + 1):
        try:
            return func()
        except Exception as e:
            if i == attempts:
                raise
```

```
            print(f"Attempt {i} failed: {e}, retrying...")
            time.sleep(delay)
```

Log Output

```
log() { echo "[$(date +'%F %T')] $*"; }
log "Deployment started"
```

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
logging.info("Deployment started")
```

Handle Errors

```
set -euo pipefail
trap 'echo "Error on line $LINENO" >&2' ERR
```

```
try:
    risky_operation()
except SpecificError as e:
    logging.error(f"Failed: {e}")
    raise
```

Read Configuration

```
source config.env    # simple key=value shell config
echo "$DB_HOST"
```

```
import yaml
config = yaml.safe_load(open("config.yml"))
```

Read Environment Variables

```
: "${AWS_REGION:=us-east-1}"   # default if unset
```

```
import os
region = os.environ.get("AWS_REGION", "us-east-1")
```

Execute External Commands

```
output=$(systemctl status nginx)
```

```
import subprocess
result = subprocess.run(["systemctl", "status", "nginx"], capture_output=True, text=True)
output = result.stdout
```

Parse JSON

```bash
value=$(echo "$json" | jq -r '.status')   # needs jq installed
```

```python
import json
data = json.loads(json_str)
value = data["status"]
```

Call APIs

```bash
curl -s -X GET https://api.example.com/status | jq '.'
```

```python
import requests
resp = requests.get("https://api.example.com/status", timeout=5)
data = resp.json()
```

Validate Input

```bash
if [[ ! "$env" =~ ^(dev|staging|prod)$ ]]; then
    echo "Invalid environment: $env" >&2
    exit 1
fi
```

```python
if env not in {"dev", "staging", "prod"}:
    raise ValueError(f"Invalid environment: {env}")
```

Return Proper Exit Codes

```bash
# 0 = success, 1-255 = failure (by convention: use distinct codes for distinct failure mod
exit 0
exit 1
```

```python
import sys
sys.exit(0)
sys.exit(1)
```

Quick Revision

- These 12 patterns cover ~80% of what real DevOps scripts do: check → retry → log → error-handle → configure → execute → parse → validate → exit.

Must Remember

1. Every script should validate its preconditions (command exists, file exists, valid input) before doing real work.
2. Retry logic belongs around anything network-dependent.
3. Exit codes are a contract — be deliberate about them.
4. `set -euo pipefail` + `trap` in Bash ≈ `try/except` + logging in Python.

Practice

1. Write a Bash pattern library file (`lib.sh`) with `log()`, `retry()`, and `require_command()` functions, sourced by other scripts.
2. Write the Python equivalent as a `utils.py` module.
3. Build a script combining 4+ of these patterns (e.g., validate input → check service → retry a health check → log → exit with proper code).

PART 5 — REAL DEVOPS MINI PROJECTS [MUST KNOW]

Each project: Problem → Architecture/Approach → Technologies → What to Build → Key Concepts Learned.

---

1. Linux System Monitoring Script

Problem: No quick way to see CPU, memory, disk, and load on a server at a glance. **Approach:** A Python (or Bash) script that collects key metrics, prints a formatted report, and optionally logs to a file/exits non-zero if thresholds are breached. **Technologies:** Python (`psutil`, `shutil`), or Bash (`top`, `free`, `df`, `uptime`). **Build:** - Collect CPU%, memory%, disk%, load average, top 5 processes by CPU. - Print a clean report (or write JSON for downstream tooling). - Exit code reflects overall health (0 = OK, 1 = warning, 2 = critical). **Key concepts:** `subprocess/psutil`, threshold logic, exit codes, structured output.

---

2. Automated Backup System

Problem: Application data/config needs regular, verified backups with retention. **Approach:** A scheduled (cron) script that archives target directories, timestamps them, uploads to S3, and deletes backups older than N days both locally and remotely. **Technologies:** Python (`shutil`, `tarfile`, `boto3`) or Bash (`tar`, `aws s3`), cron. **Build:** - Create a timestamped `tar.gz` of a source directory. - Upload to S3 with a dated key prefix. - Delete local/remote backups older than the retention window. - Log success/failure; alert (e.g., email/Slack webhook) on failure. **Key concepts:** file archiving, S3 upload/lifecycle logic, retention policies, cron scheduling, idempotency.

---

3. Log Analyzer

Problem: Manually grepping through logs during incidents is slow and error-prone. **Approach:** A script that parses log files, extracts error patterns/timestamps/IPs via regex, aggregates counts, and outputs a summary report (top errors, error rate over time). **Technologies:** Python (`re`, `collections.Counter`) or Bash (`grep`, `awk`, `sort | uniq -c`). **Build:** - Parse structured or semi-structured log lines. - Count occurrences per error type / per hour. - Flag anomalies (e.g., error rate spike vs baseline). - Output a summary (console table or JSON/CSV). **Key concepts:** regex, aggregation, `Counter`, log parsing patterns, basic anomaly thresholds.

---

4. AWS EC2 Automation

Problem: Manually starting/stopping/tagging EC2 instances is slow and inconsistent. **Approach:** A boto3 CLI tool to list, filter by tag, start/stop, and report on EC2 instances — e.g., stop all `Environment=dev` instances outside business hours to cut cost. **Technologies:** Python, `boto3`, `argparse`. **Build:** - `list` command: show instances filtered by tag/state. - `stop-dev` command: stop all dev-tagged running instances (with `--dry-run`). - Schedule via cron or (better) a Lambda + EventBridge rule. **Key concepts:** boto3 filters, pagination, `ClientError` handling, `--dry-run` safety pattern, cost optimization.

---

5. S3 Backup Automation

Problem: Need a lightweight, reliable way to sync a directory to S3 as an offsite backup. **Approach:** A script that walks a local directory, uploads new/changed files to S3 (checking ETags/hashes to avoid redundant uploads), and logs the sync summary. **Technologies:** Python (`boto3`, `hashlib`) or Bash (`aws s3 sync`). **Build:** - Compare local file hash vs S3 object ETag before uploading (skip unchanged files). - Upload changed/new files; optionally mirror deletions with `--delete` (Bash) or manual diffing (Python). - Summary report: files uploaded, skipped, failed. **Key concepts:** S3 API basics, hashing for change detection, sync vs full-copy strategies.

---

6. Docker Cleanup Automation

Problem: Docker hosts accumulate stopped containers, dangling images, and unused volumes, consuming disk space. **Approach:** A scheduled script that safely prunes unused Docker resources and reports space reclaimed. **Technologies:** Bash (`docker container/image/volume prune`) or Python (`subprocess` wrapping the Docker CLI, or the `docker` SDK). **Build:** - Report current Docker disk usage (`docker system df`). - Prune stopped containers, dangling images, unused volumes (with a safe filter, e.g., don't touch anything used in the last 24h). - Log space reclaimed; run weekly via cron. **Key concepts:** Docker resource lifecycle, safe automated cleanup, cron scheduling.

---

7. Service Health Checker

Problem: Need to know immediately if a critical service goes down, and optionally self-heal. **Approach:** A script (running via cron or as a systemd timer) that checks a service's status, restarts it if down, and alerts if the restart fails. **Technologies:** Bash (`systemctl`) or Python (`subprocess`/`psutil`), optional Slack/webhook alerting. **Build:** - Check `systemctl is-active`. - If inactive: attempt restart, wait, re-check. - If still down after restart: send an alert (webhook/email) and exit non-zero. - Log every check (for later analysis of flapping services). **Key concepts:** self-healing automation, alerting integration, idempotent restart logic, cron/systemd timers.

---

8. Website Uptime Monitor

Problem: Need to know when a public endpoint goes down or gets slow, without a full APM tool. **Approach:** A script that periodically checks HTTP status + response time for a list of URLs, logs results, and alerts on failures or degraded latency. **Technologies:** Python (`requests`) or Bash (`curl -w`), cron, optional webhook alert. **Build:** - Read a list of URLs from a config file. - For each: check status code + response time, with timeout + retry. - Log results (CSV/JSON) for historical uptime tracking. - Alert if a URL fails N consecutive checks. **Key concepts:** HTTP health checks, timeout/retry patterns, consecutive-failure alerting logic, historical logging.

---

9. Server Inventory Collector

Problem: No central, up-to-date view of what's installed/running across a fleet of servers. **Approach:** A script deployed to each server (via SSH/Ansible/cron) that collects OS version,

installed packages, running services, disk/memory, and reports it centrally (file, S3, or an API). **Technologies:** Python (`platform`, `psutil`, `subprocess`) or Bash, plus `ssh`/`scp` or an S3 upload for centralization. **Build:** - Collect hostname, OS, kernel version, uptime, disk/memory, key installed packages, running services. - Format as JSON. - Upload/send to a central location (S3 bucket keyed by hostname+date, or POST to an internal API). **Key concepts:** structured data collection, remote execution (SSH), centralizing fleet data, JSON as the interchange format.

---

10. Automated Deployment Script

Problem: Manual deployments are error-prone and inconsistent across environments. **Approach:** A script that automates the full deploy sequence: pull code, build, deploy (Docker/systemd), run health checks, and roll back automatically on failure. **Technologies:** Bash (`git`, `docker compose`, `curl`) or Python (orchestrating the same via `subprocess`), `set -euo pipefail`. **Build:** - Step 1: pull latest code/image for the target version. - Step 2: build/deploy (docker compose up -d –build, or systemd restart). - Step 3: run a post-deploy health check (HTTP + service status). - Step 4: if health check fails, roll back to the previous known-good version and alert. - Log every step with timestamps; support `--dry-run`. **Key concepts:** deployment orchestration, health-check-gated rollout, rollback strategy, `set -euo pipefail`, logging every step for audit/debugging.

Quick Revision

- All 10 projects share the same skeleton: **gather info → decide/act → verify → log/alert → (optionally) roll back**.
- Build these roughly in order — each one reuses patterns from Part 4.

Must Remember

1. Every project should support `--dry-run` if it can modify/delete anything.
2. Every project should log clearly and exit with a meaningful code.
3. Alerting (even a simple webhook) turns a “script” into “automation you can trust.”
4. Start in Bash for the simplest ones (health checker, Docker cleanup); move to Python once JSON/AWS/complex logic is involved.

Practice

- Pick 3 of these projects and actually build them end-to-end this week — that’s more valuable than reading all 10 descriptions.

PART 6 — QUICK REVISION SHEETS

Python Cheat Sheet [MUST KNOW]

Syntax basics

```
x = 5; s = "text"; f"{x} {s}"
if x > 0: ...
elif x == 0: ...
else: ...
for i in range(5): ...
while cond: ...
```

Data structures

```
[1,2,3]             # list - mutable, ordered
(1,2,3)              # tuple - immutable, ordered
{1,2,3}              # set - unique, unordered
{"k":"v"}            # dict - key/value
[x for x in items if cond]        # list comprehension
{k:v for k,v in items}             # dict comprehension
```

Loops

```
for i, v in enumerate(items): ...
for a, b in zip(list1, list2): ...
break; continue
```

Functions

```
def f(a, b=10, *args, **kwargs):
    return a + b
lambda x: x * 2
```

File handling

```
with open("f.txt") as f: data = f.read()
from pathlib import Path
Path("f.txt").exists()
```

Exceptions

```
try:
    ...
except SpecificError as e:
    ...
finally:
    ...
raise ValueError("msg")
```

OS automation

```
import subprocess
r = subprocess.run(["cmd","arg"], capture_output=True, text=True, check=True)
r.stdout; r.returncode
import os
os.environ.get("VAR", "default")
```

JSON

```
import json
json.loads(s); json.dumps(obj, indent=2)
json.load(f); json.dump(obj, f)
```

APIs

```
import requests
r = requests.get(url, timeout=5); r.raise_for_status(); r.json()
```

Boto3

```
import boto3
ec2 = boto3.client("ec2")
ec2.describe_instances()
s3 = boto3.client("s3")
s3.upload_file(local, bucket, key)
```

Logging

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("msg"); logging.error("msg"); logging.exception("msg")
```

Argparse

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--source", required=True)
p.add_argument("--dry-run", action="store_true")
args = p.parse_args()
```

Bash Cheat Sheet [MUST KNOW]

Variables

```
var="value"
echo "$var" "${var}"
result=$(command)
count=$((count + 1))
```

Conditions

```
if [ "$a" == "$b" ]; then ...
elif [[ "$a" -gt 10 ]]; then ...
else ... fi

-z / -n     # empty / not empty string
-eq -ne -gt -lt -ge -le   # numeric
-f -d -e -x               # file / dir / exists / executable
```

Loops

```
for x in a b c; do ...; done
for (( i=0; i<5; i++ )); do ...; done
while [ cond ]; do ...; done
until [ cond ]; do ...; done
```

Functions

```
myfunc() {
    local var=$1
    echo "$var"
    return 0
}
result=$(myfunc "arg")
```

Arguments

```
$0 $1 $2 "$@" $# $?
```

Pipes & redirection

```
cmd1 | cmd2
cmd > out.log
cmd >> out.log
cmd 2> err.log
cmd > out.log 2>&1
cmd1 && cmd2      # run cmd2 only if cmd1 succeeds
cmd1 || cmd2      # run cmd2 only if cmd1 fails
cmd &              # background
```

Important commands

```
ls -la; cd; pwd; mkdir -p; cp -r; mv; rm -rf
cat; less; head -n; tail -f
grep -r -i -v -c -E; find -name -mtime -delete
awk '{print $1}'; sed 's/a/b/g'; sort; uniq -c; cut -d',' -f1; xargs; wc -l
chmod +x; chown user:group
ps aux; top -bn1; df -h; du -sh; free -m; uptime
systemctl status/restart/is-active; journalctl -u <svc> -f
curl -s -o /dev/null -w "%{http_code}"; wget; tar -czf/-xzf; ssh; scp; rsync -avz
```

Exit codes

```
exit 0    # success
exit 1    # generic failure
$?        # check last exit code
set -euo pipefail   # fail fast script safety net
```

Permissions

```
chmod 755 script.sh   # rwxr-xr-x
chmod +x script.sh
chmod 600 secrets.env  # owner read/write only
```

Linux Commands Cheat Sheet [MUST KNOW]

Category	Must-know commands
Navigation	`ls`, `cd`, `pwd`, `mkdir -p`
Files	`cp -r`, `mv`, `rm -rf`, `touch`, `find`
Viewing	`cat`, `less`, `head`, `tail -f`

Category	Must-know commands
Text processing	`grep`, `awk`, `sed`, `sort`, `uniq -c`, `cut`, `wc -l`, `xargs`, `tr`, `tee`
Permissions	`chmod`, `chown`
Processes	`ps aux`, `top -bn1`, `pgrep`, `kill`
Resources	`df -h`, `du -sh`, `free -m`, `uptime`
Services	`systemctl status/restart/is-active`, `journalctl -u -f`
Network	`curl -s -o /dev/null -w "%{http_code}"`, `wget`, `ssh`, `scp`, `rsync -avz`
Archives	`tar -czf`, `tar -xzf`

Daily-driver combos worth memorizing:

```
grep "ERROR" app.log | wc -l
df -h | awk '{print $5, $6}'
find /var/log -mtime +7 -name "*.log"
sort access.log | uniq -c | sort -rn | head
tail -f /var/log/syslog
curl -s -o /dev/null -w "%{http_code}\n" https://example.com
systemctl is-active --quiet nginx && echo up || echo down
```